Student	Container Runtime_Mai_Hung
Started	10/18/2026, 11:45:48 PM
Submitted	10/18/2026, 11:52:48 PM

MODULE 1 — CONTAINERS

Container Runtime Architecture

WHY THIS MATTERS

THE PROBLEM

Software works on a developer's machine but breaks in production because environments differ — OS versions, library paths, runtime configs. Traditional deploys carry this risk.

THE SOLUTION

Containers bundle the application with everything it needs (dependencies, config, runtime) into a single portable image that runs identically on any machine with a container runtime.

REAL-WORLD USE

Every major cloud platform (AWS ECS, Google Kubernetes Engine, Azure Container Apps) runs workloads as containers. Docker is the industry-standard build and run toolchain.

YOUR GOAL

Understand the layered architecture from code to hardware so you can reason about where failures happen, why containers are lighter than VMs, and how isolation is enforced.

DESIGN CANVAS — DRAG COMPONENTS IN THE CORRECT ORDER

1. Application Code
2. Dockerfile (Image Definition)
3. Container Image (Built Artifact)
4. Container Runtime (Docker Engine)
5. OS Namespaces and cgroups (Isolation)
6. Host OS Kernel

7. Physical / Cloud Hardware

STATUS: PASS / COMPLETED

Container runtime architecture verified. The stack correctly models how application code is packaged into an image by a Dockerfile, executed by the container runtime, isolated by the OS via namespaces and cgroups, all running on the host kernel atop physical hardware.

© 2026 Clark Ngo